

JSS SCIENCE AND TECHNOLOGY UNIVERSITY

JSS Technical Institutions Campus, Mysuru – 570006



“Fake news detector”

Mini project report submitted in partial fulfillment of curriculum prescribed for the Web Technology (22CS620) course for the award of the degree of

**BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING**

By

Roll Number	USN	Name
24	01JST23UCS093	Nitish K
20	01JST23UCS069	M Arun Kumar
10	01JST23UCS034	Deepak L
15	01JST23UCS058	Karthik M S

Under the Guidance of

Savitha N

Associate Professor, Dept. Of CS & E,
SJCE JSS STU Mysore

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
2026-27**

JSS SCIENCE AND TECHNOLOGY UNIVERSITY

JSS Technical Institutions Campus, Mysuru – 570006



CERTIFICATE

This is to certify that the work entitled “**Fake news detector**” is a bonified work carried out by **Nitish K, Deepak L, Karthik M S, and M Arun kumar** in partial fulfillment of the award of the degree of **Bachelor of Engineering in Computer Science and Engineering of SJCE, JSS Science and Technology, Mysuru during the year 2026**. It is certified that all corrections / suggestions indicated during CIE have been incorporated in the report. The mini project report has been approved as it satisfies the academic requirements in respect of mini project work prescribed for the Web Technology (22CS620) course.

Course In Charge and Guide

Savitha N

Associate Professor
Dept. of CS & E, SJCE
JSS STU Mysore

Place: Mysore

Date : 12-05-2026

ABSTRACT

Fake news has become a major problem in today's digital world due to the rapid growth of social media and online news platforms. False information spreads quickly and influences public opinion, causing confusion and misinformation. Manually verifying every news article is difficult and time-consuming. Therefore, an automated system is required to detect fake news efficiently.

This project presents a Fake News Detection System using Machine Learning and Natural Language Processing (NLP) techniques. The system analyzes news articles and classifies them as fake or real. The dataset used for this project is collected from Kaggle and contains fake and real news articles.

The project includes data preprocessing, TF-IDF vectorization, feature extraction, model training, prediction, and evaluation. Logistic Regression is used as the primary classification algorithm because of its high accuracy and efficient performance. The developed system helps reduce misinformation spread and improves awareness regarding fake news detection using Artificial Intelligence.

The developed system is a user-friendly, interactive web application that makes advanced machine learning accessible via a browser-based interface. It utilizes HTML, CSS, and JavaScript to build a responsive, intuitive frontend for users to easily input news articles. The backend is powered by Python and the Flask framework, which seamlessly connects the user interface to the trained Logistic Regression model for real-time predictions.

Sl. No.	Title	Page No.
1	Introduction	1
2	Objectives	2
3	Problem Statement	3
4	Software Requirements	4
5	Dataset Description	5
6	Methodology	6
6.1	Data Collection	6
6.2	Data Labelling	7
6.3	Data Merging	7
6.4	Data Preprocessing	8
6.5	Feature Extraction using TF-IDF	9
6.6	Dataset Splitting	10
6.7	Model Training	10
6.8	Prediction	11
6.9	Model Evaluation	11
7	Implementation	12
8	Project Workflow	13
9	Results	14
10	Advantages	15
11	Applications	16
12	Future Enhancements	17
13	Conclusion	18
14	References	19

1. INTRODUCTION

Fake news refers to false or misleading information presented as genuine news. With the growth of online platforms and social media applications, fake news spreads rapidly and affects society, politics, businesses, and individuals.

Machine Learning and Natural Language Processing provide effective techniques to automatically analyze textual content and identify whether news is fake or real. This project focuses on building a Web app model capable of detecting fake news articles based on textual features.

The system uses text preprocessing methods and TF-IDF vectorization to convert textual data into numerical format. A classification algorithm is trained using labeled news data to predict the authenticity of news articles.

2. OBJECTIVES

The main objective of this project is to develop a Machine Learning system that automatically identifies whether a news article is fake or real.

Specific Objectives

- Detect misleading or false news articles automatically
- Reduce the spread of misinformation on digital platforms
- Analyze textual content of news articles
- Convert textual data into machine-understandable format
- Train a machine learning model for classification
- Improve awareness regarding fake news detection using AI
- Evaluate model performance using machine learning metrics

3. PROBLEM STATEMENT

With the rapid growth of social media and online news platforms, fake news spreads quickly and affects public opinion. Manually verifying every news article is difficult and time-consuming. Therefore, an automated fake news detection system is required to classify news articles as fake or real efficiently.

4. SOFTWARE REQUIREMENTS

Requirement	Purpose
Python 3.x	Programming Language
Jupyter Notebook / VS Code	Development Environment
Pandas	Data Handling
NumPy	Numerical Computation
Scikit-learn	Machine Learning
NLTK	Natural Language Processing
Matplotlib	Data Visualization
Seaborn	Graphical Analysis
HTML, CSS, Javascript	Web design
Python Flask	Backend to handle API requests

5. DATASET DESCRIPTION

The dataset used in this project is obtained from Kaggle.

Data set

Link:

<https://www.kaggle.com/datasets/emineyetm/fake-news-detection-datasets>

The dataset contains two CSV files:

- **Fake.csv** → Fake news articles
- **True.csv** → Real news articles

Dataset Features

- News Title
- News Text
- Subject/Category
- Publication Date

The dataset is used for training and testing the machine learning model.

6. PROJECT ARCHITECTURE

The proposed fake news detection system is built upon a robust **client-server architecture** designed to bridge high-performance machine learning with an intuitive user interface. The **backend** leverages the **Flask** micro-framework, which serves as the central orchestration layer, managing API requests and facilitating seamless communication between the frontend and the **Machine Learning model**. This model, integrated into the Flask environment, performs the core computational analysis to classify news veracity based on input data. On the **frontend**, a modular design approach is employed using **HTML5** for structural components and **CSS3** for aesthetic styling and responsive layouts. Client-side interactivity and asynchronous data handling are governed by **JavaScript** ensuring a dynamic user experience where news articles can be submitted and verified in real-time without requiring full page reloads. The project architecture is given below.

Technical Breakdown

Component	Technology	Purpose
Logic & Processing	Flask (Python)	Routes data, handles logic, and serves the ML model.
Intelligence	ML Model	Analyzes text patterns to predict the probability of fake news.
Structure	HTML	Defines the skeleton of the landing page and navigation bar.
Styling	CSS	Ensures a professional, user-friendly interface design.
Interactivity	JavaScript	Handles DOM manipulation and sends fetch requests to the backend.

Communication Flow Summary

Action	Direction	Data Format
Request (POST)	Frontend=>Backend	JSON: { "text": "..."}
Processing	Backend Internal	String => Vector => Label
Response	Backend=>Frontend	JSON: { "prediction": "...", "confidence": "..."}

7. PROJECT WORKFLOW

News Dataset via Text box in web app



Data Preprocessing



TF-IDF Vectorization



Train/Test Split



Logistic Regression Model



Prediction



Fake / Real Output

The system follows a linear data processing pipeline designed to transform unstructured text into a definitive classification. The workflow begins at the user interface, where raw news content is captured via a web-based text input. This data is immediately passed to the backend for preprocessing, a critical phase where the noise is filtered out through techniques like tokenization, stop-word removal, and stemming. To make this text intelligible to the machine, it undergoes TF-IDF (Term Frequency-Inverse Document Frequency) Vectorization, which converts words into a weighted numerical matrix based on their statistical significance. Following a standardized Train/Test Split to ensure model validity, the processed vectors are fed into a Logistic Regression model. This algorithm calculates the probability of the input's authenticity, culminating in a final Prediction that identifies the news as either Fake or Real, which is then rendered back to the user interface.

8. IMPLEMENTATION

- **System Integration: The Request-Response Lifecycle**

The connection between the frontend and backend is established through a RESTful API architecture, specifically using the HTTP POST method to handle data transmission. When a user enters news text into the frontend interface, the JavaScript logic (`predict.js`) captures the input and packages it into a JSON object. It then initiates an asynchronous `fetch` request to the Flask server's `/predict` endpoint. This "POST" request is essential because it allows the system to send large blocks of text securely within the request body, rather than appending them to the URL.

- **Backend Handling and Model Execution**

On the server side, the **Flask API** remains in a listening state, ready to intercept incoming requests. Once the `/predict` route receives the JSON payload, the backend performs the following steps:

- **Parsing:** It extracts the raw string from the text field of the request.
- **Processing:** The text is passed through the `wordopt` function for cleaning and then transformed via the TF-IDF Vectorizer.
- **Inference:** The pre-trained Logistic Regression model calculates the probability of the news being "Fake" (Class 0) or "Real" (Class 1).
- **Response:** Flask encapsulates these findings—including the final prediction label and the specific confidence percentages—into a JSON response, which is sent back to the client with a `200 OK` status code.

- **Frontend Rendering and Dynamic Updates**

The final phase occurs when the JavaScript `fetch` promise resolves with the backend's data. The frontend doesn't refresh the entire page; instead, it uses DOM manipulation to update specific UI elements dynamically. The script reads the prediction result and confidence scores, then updates the result Box CSS classes to change colors (e.g., red for "Fake," green for "Authentic"). This seamless "handshake" between the Python-based analytical engine and the JavaScript-based presentation layer ensures that the machine learning insights are delivered to the user in a highly interactive and responsive environment.

9. RESULTS

The Logistic Regression model successfully classifies fake and real news articles with high accuracy. TF-IDF vectorization improves text representation and enhances model performance.

The system effectively predicts the authenticity of news articles using textual features.

Example Output:

News: "Government launches new policy"

Prediction: Real News

10. ADVANTAGES

- Fast fake news identification
- Reduces misinformation spread
- Saves manual verification effort
- Improves online news reliability
- Useful for social media monitoring

11. APPLICATIONS

- Social Media Platforms
- News Websites
- Online Content Verification
- Journalism and Media Monitoring
- Fact-Checking Systems

12. CONCLUSION

The development and implementation of this Fake News Detection system demonstrate a successful integration of modern web technologies and machine learning to address the pervasive issue of digital misinformation. By leveraging a lightweight Flask API on the backend and a modular HTML5/CSS3/JavaScript frontend, the architecture provides a highly responsive, user-friendly platform for real-time text analysis. The underlying analytical pipeline—combining TF-IDF Vectorization with a Logistic Regression classifier—proves that robust, mathematically sound machine learning models can be effectively deployed within a web environment to yield high-confidence predictions without introducing computational latency.

Ultimately, this project highlights the importance of separating concerns in software engineering, where complex data preprocessing and model inference are isolated on the server, while the client interface remains fast and interactive. As misinformation continues to evolve in complexity, the scalable design of this application provides a solid foundation for future enhancements. Prospective iterations can easily expand upon this framework by integrating deep learning capabilities, implementing automated web-scraping for cross-referencing source credibility, or connecting databases to track recurring propagation patterns of fake news.